Akuna OA：Future Stock Prices（Java 解法）

题意

给定无序的 [Stock, Date(MM/DD/YYYY), Price] 列表，初始 $1000，可买卖碎股、无手续费、不能做空，只能在该股票有报价的日期交易。求最大利润，四舍五入到整数美元。

思路

- 碎股 + 无手续费 ⇒ 资金变化全是乘法，任何时刻 all-in 一个最优选择即可，分仓不会更好。
- 按日期排序，维护 cash = 当前能持有的最多现金，shares[s] = 当前能持有的股票 s 的最多股数。
- 同一天先“卖”：cash = max(cash, shares[s]·p)；再“买”：shares[s] = max(shares[s], cash/p)。这样允许同一天卖 A 买 B。
- 答案为 round(cash − 1000)。时间 O(n log n)（排序），空间 O(n)。

最优解

```
public static long maxProfit(java.util.List<java.util.List<String>> prices) {
    int n = prices.size();
    String[] stock = new String[n];
    int[] date = new int[n];
    double[] price = new double[n];
    Integer[] order = new Integer[n];
    for (int i = 0; i < n; i++) {
        java.util.List<String> row = prices.get(i);
        stock[i] = row.get(0).trim();
        String[] mdy = row.get(1).trim().split("/");           // MM/DD/YYYY
        date[i] = Integer.parseInt(mdy[2].trim()) * 10000
                + Integer.parseInt(mdy[0].trim()) * 100
                + Integer.parseInt(mdy[1].trim());
        price[i] = Double.parseDouble(row.get(2).trim());
        order[i] = i;
    }
    java.util.Arrays.sort(order, (a, b) -> Integer.compare(date[a], date[b]));

    double cash = 1000.0;                                      // best cash we can hold now
    java.util.Map<String, Double> shares = new java.util.HashMap<>(); // best #shares of each stock
    for (int i = 0; i < n; ) {
        int j = i;
        while (j < n && date[order[j]] == date[order[i]]) j++;  // [i, j) = same day
        double newCash = cash;                                 // 1) sell anything priced today
        for (int k = i; k < j; k++) {
            int t = order[k];
            Double s = shares.get(stock[t]);
            if (s != null) newCash = Math.max(newCash, s * price[t]);
        }
        cash = newCash;
        for (int k = i; k < j; k++) {                          // 2) buy with best cash today
            int t = order[k];
            if (price[t] > 0) shares.merge(stock[t], cash / price[t], Math::max);
        }
        i = j;
    }
    return Math.round(cash - 1000.0);
}
```

暴力解（对拍用）

```
// 暴力：枚举所有交易序列。每笔交易 = 选一只股票的两个报价（买 d1 < 卖 d2），
// 下一笔买入日期 >= 上一笔卖出日期（允许同一天卖出再买入）。指数复杂度，仅用于小数据对拍。
public static double bruteForce(java.util.List<java.util.List<String>> prices) {
    int n = prices.size();
    String[] st = new String[n]; int[] d = new int[n]; double[] p = new double[n];
    for (int i = 0; i < n; i++) {
        st[i] = prices.get(i).get(0).trim();
        String[] m = prices.get(i).get(1).trim().split("/");
        d[i] = Integer.parseInt(m[2]) * 10000 + Integer.parseInt(m[0]) * 100 + Integer.parseInt(m[1]);
        p[i] = Double.parseDouble(prices.get(i).get(2).trim());
    }
    double[] best = {1000.0};
    java.util.function.BiConsumer<Integer, Double>[] dfs = new java.util.function.BiConsumer[1];
    dfs[0] = (from, cash) -> {
        best[0] = Math.max(best[0], cash);                      // 选择就此停手
        for (int b = 0; b < n; b++) {
            if (d[b] < from) continue;
            for (int s = 0; s < n; s++) {
                if (st[s].equals(st[b]) && d[s] > d[b]) dfs[0].accept(d[s], cash / p[b] * p[s]);
            }
        }
    };
    dfs[0].accept(Integer.MIN_VALUE, 1000.0);
    return best[0] - 1000.0;
}
```

对拍结果

- Example 1：输出 60（期望 60）；Example 2：输出 127（期望 127）。
- 随机 20000 组（1–4 只股票、1–6 个日期、最多 9 条报价、输入打乱顺序）：最优解与暴力解 0 处不一致。
- n = 100000 规模测试：约 240 ms。